Proyecto Front-End: Parte 4 (Avance)

Estudiantes: Di Fini, Zaira y Ferreyra, Evelin

Materia: Aplicaciones Web I

Profesor: Senn, Horaldo Andres

Índice

Organización..... 3
Registro de Cambios y Modificaciones (index.html)..... 4
Desarrollo y Codificación de la Vista Inicio (index.html, index.css y Módulos JS)..... 4
Fundamentación Técnica de JavaScript (index.js)..... 5
Registro de Cambios y Modificaciones (perfil.html)..... 6
Desarrollo y Codificación de la Vista Perfil (perfil.html, perfil.css y perfil.js)..... 7
Fundamentación Técnica de JavaScript (perfil.js)..... 9
Registro de Cambios y Modificaciones (artistas.html)..... 10
Desarrollo y Codificación de la Vista Artistas (artistas.html, artistas.css y artistas.js)..... 11
Fundamentación Técnica de Propiedades y Métodos JavaScript (MDN / Documentación Oficial)..... 12
Registro de Cambios y Modificaciones (contacto.html)..... 13
Desarrollo y Codificación de la Vista Contacto (contacto.html, contacto.css y contacto.js)..... 14
Fundamentación Técnica de Propiedades y Métodos (MDN / TID)..... 14
Fuentes Consultadas..... 16

Organización

El repositorio del proyecto está estructurado de manera limpia y modular para facilitar la escalabilidad, el mantenimiento y la correcta navegación entre recursos estáticos:

- **Archivos HTML principales:** Ubicados en la raíz del proyecto (index.html, perfil.html, artistas.html, contacto.html), garantizando rutas relativas limpias y directas.
- **Carpeta de recursos multimedia (/recursos):** Almacena imágenes optimizadas para tarjetas, portadas de artistas, Fan Packs e iconografía general de la plataforma.
- **Hojas de estilos:** Los archivos CSS se encuentran organizados dentro de la carpeta correspondiente, centralizando los estilos generales en estilos.css y utilizando hojas específicas para las distintas vistas.
- **Módulos JavaScript:** Los archivos se encuentran organizados para separar las funcionalidades de cada sección y facilitar su mantenimiento.
- **Distribución de tareas y sincronización:**
    - ➔ El desarrollo de las páginas index.html y perfil.html estuvo a cargo de Zaira Di Fini, quien también programó los archivos JavaScript necesarios para ambas vistas: index.js, perfil.js, catalogo.js, eventosMusicales.js, detalleEventos y detalleCatalogo,
    - ➔ Las vistas contacto.html y artistas.html fueron desarrolladas por Evelin Ferreyra, incluyendo sus respectivos archivos JavaScript: artistas.js, contacto.js, datosArtistas.js, funcionesArtistas.js
    - ➔ Asimismo, se realizó una revisión y corrección cruzada de los módulos trabajados para mejorar la integración y el funcionamiento general del proyecto.

Registro de Cambios y Modificaciones (index.html)

En relación con la maqueta estructural desarrollada en el TP3, se realizaron modificaciones en el código HTML y en la hoja de estilos externa index.css para adaptar la vista de Inicio a la nueva organización modular del proyecto y permitir la visualización dinámica de los eventos musicales.

- **Incorporación del contenedor del módulo de eventos:** Se agregó el elemento #contenedor-modulo-eventos, que funciona como punto de inserción del contenido generado dinámicamente por el módulo eventosMusicales.js.
- **Actualización de los accesos directos:** Se incorporaron los enlaces «Ver eventos top», que dirige a la sección de eventos, y «Ver artistas del momento», que permite acceder directamente a artistas.html.
- **Incorporación de la sección de detalle de eventos:** Se agregó una estructura HTML específica para mostrar la información del evento seleccionado, incluyendo imagen, artista, nombre, descripción, fecha, horarios, puntos de acceso, elementos permitidos, setlist y retiro del Fan Pack.
- **Visualización condicional del detalle:** Se incorporó la clase seccion-oculta para mantener oculto el detalle del evento hasta que el usuario seleccione uno de los eventos disponibles.

Desarrollo y Codificación de la Vista Inicio (index.html, index.css y Módulos JS)

La página principal se organizó para separar la estructura visual de la lógica de funcionamiento, permitiendo incorporar el módulo de eventos y acceder al detalle de cada uno mediante módulos JavaScript independientes.

- **Banner principal de bienvenida:** Presenta el título «Bienvenido a SoundDay» y el subtítulo «Descubre y planifica tus próximos shows». Se

aplicó un fondo con degradado diagonal, bordes redondeados y un elemento decorativo lateral generado mediante CSS.

- **Sección de anuncios y noticias:** Se estructuró mediante tarjetas informativas distribuidas con CSS Grid. Se incorporaron transiciones y efectos visuales al pasar el cursor sobre ellas.
- **Módulo de eventos musicales (eventosMusicales.js):** Se utiliza para inicializar e insertar dinámicamente el contenido de eventos dentro del contenedor #contenedor-modulo-eventos, evitando definir directamente esta estructura en index.html.
- **Módulo de detalle de eventos (detalleEventos.js):** Se encarga de gestionar la apertura y visualización de la información del evento seleccionado. Se utiliza desde Inicio mediante la función abrirDetalleEvento, a la que se le proporciona el identificador del evento y la función encargada de mostrar su sección de detalle.

Fundamentación Técnica de JavaScript (index.js)

Para la inicialización y coordinación de la vista Inicio se utilizaron las siguientes características de JavaScript y métodos de la API del Modelo de Objetos del Documento (DOM):

- **Sentencia import:** Se utiliza para importar las funciones inicializarModuloEventos desde eventosMusicales.js y abrirDetalleEvento desde detalleEventos.js, permitiendo utilizar funcionalidades externas dentro de index.js.
- **Método document.querySelector():** Se utiliza para seleccionar los elementos HTML #contenedor-modulo-eventos y #detalles-evento, obteniendo referencias a los elementos donde se inicializa el módulo de eventos y donde se muestra el detalle seleccionado.
- **Modelo de Objetos del Documento (DOM):** Se utiliza para acceder y modificar elementos de la estructura HTML. En este caso, permite controlar la visualización de la sección de detalle mediante seccion.style.display y seccion.classList.remove().
- **Definición de funciones:** Se definió la función mostrarSeccion(seccion), que recibe como parámetro el elemento que debe mostrarse. También verifica

que la sección exista antes de modificarla y, si no se encuentra, muestra un mensaje de error mediante console.error().

- **Método addEventListener():** Se utiliza para escuchar el evento personalizado abrirDetalleEvento en el contenedor de eventos. Cuando se recibe dicho evento, se ejecutan las instrucciones definidas en la función callback.
- **Funciones flecha:** Se utilizan para definir la función callback que se ejecuta cuando se recibe el evento abrirDetalleEvento.
- **Método scrollIntoView():** Se utiliza para desplazar automáticamente la página hasta la sección seleccionada. Se configura con behavior: "smooth" para realizar un desplazamiento suave y block: "start" para ubicar el elemento al comienzo del área visible.

Registro de Cambios y Modificaciones (perfil.html)

En relación con la maqueta estructural desarrollada en el TP2, se realizaron modificaciones en el código HTML y en la hoja de estilos externa perfil.css para adaptar la vista de Perfil a la nueva organización modular del proyecto y permitir la gestión dinámica de artistas seguidos, eventos guardados y reservas de Fan Packs.

- **Incorporación de secciones dinámicas:** Se agregaron contenedores específicos para mostrar los artistas seguidos, el catálogo de Fan Packs, los eventos guardados y las reservas del usuario. Su contenido se genera y actualiza mediante JavaScript.
- **Visualización condicional de las secciones:** Se incorporó la clase sección-oculta para mantener ocultas las secciones de artistas seguidos, catálogo de Fan Packs y detalles de eventos y productos hasta que el usuario acceda mediante los enlaces o botones correspondientes.
- **Modificación de la sección de eventos:** Se reorganizó la sección «Mis próximos eventos» para mostrar los eventos guardados por el usuario, incorporando accesos para consultar sus detalles o quitarlos de favoritos.
- **Actualización de la tabla de reservas:** Se modificó la presentación de «Mis Fan Packs reservados» para mostrar inicialmente un mensaje cuando no existen reservas, junto con un enlace para acceder al catálogo de Fan Packs.

- **Incorporación del catálogo de Fan Packs:** Se agregó una sección para visualizar los productos disponibles, consultar sus detalles y filtrarlos por género musical.
- **Incorporación del detalle de Fan Packs:** Se agregó una estructura HTML específica para presentar la información del producto seleccionado, incluyendo imagen, artista, nombre, descripción, precio, contenido, modalidad de retiro y estado de la reserva.
- **Incorporación del detalle de eventos:** Se agregó una estructura HTML para mostrar la información completa del evento seleccionado, incluyendo imagen, artista, nombre, descripción, fecha, horarios, puntos de acceso, elementos permitidos, setlist y retiro del Fan Pack.
- **Reorganización de las vistas de detalle**: Se incorporaron estructuras compartidas para presentar los detalles de eventos y Fan Packs mediante tarjetas principales y paneles de información, manteniendo una organización visual uniforme.

Desarrollo y Codificación de la Vista Perfil (perfil.html, perfil.css y perfil.js)

La vista Perfil se organizó para separar la estructura visual de la lógica de funcionamiento, permitiendo gestionar los artistas seguidos, consultar los eventos guardados y administrar las reservas de Fan Packs mediante JavaScript.

- **Gestión de artistas seguidos (datosArtistas.js y perfil.js):** Se utiliza el arreglo artistasData, definido en datosArtistas.js, como fuente de información de los artistas disponibles en la plataforma. Este arreglo contiene sus datos, como identificador, nombre, género, tour, fecha, imágenes, descripción, recinto, horarios, accesos, elementos permitidos, setlist y Fan Pack asociado. En perfil.js se recuperan los identificadores de los artistas seguidos, almacenados en localStorage bajo la clave artistasSeguidos, y se relacionan con los objetos de artistasData para mostrar su información. Se incorporaron filtros por género musical (Pop, Rock y Rap) para visualizar los artistas correspondientes a cada selección.
- **Catálogo de Fan Packs (catalogo.js):** Se define el arreglo fanPacks con los productos disponibles y sus características. Este módulo proporciona los

datos necesarios para generar las tarjetas del catálogo y las funciones para consultar y administrar las reservas.

- **Gestión centralizada de reservas (catalogo.js y perfil.js):** Se centralizaron las operaciones de reservas en catalogo.js, mediante las funciones obtenerReserva(), obtenerReservas(), agregarFanPack(), quitarFanPack() y quitarUnidadFanPack(). Desde perfil.js se utilizan estas funciones para agregar productos, quitar reservas y actualizar la tabla correspondiente. De esta manera, se evita duplicar la lógica de almacenamiento en distintos módulos.
- **Almacenamiento de reservas (catalogo.js):** Las reservas se guardan en localStorage utilizando una clave independiente para cada Fan Pack, con el formato reservaFanPack_ID. Los datos se almacenan como una cadena de texto que contiene cantidad, precio y modalidad de retiro, separados por el carácter |.
- **Detalle de Fan Packs (detalleCatalogo.js):** Se utilizan las funciones abrirDetalleFanPack() y actualizarVistaDetalle() para mostrar la información del producto seleccionado y actualizar su estado de reserva. Este módulo consulta los datos y las reservas mediante las funciones centralizadas de catalogo.js.
- **Gestión de artistas y catálogo (artistas.js y funcionesArtistas.js):** Se utilizan las funciones renderizarArtistas() y filtrarArtistas() para generar las tarjetas de artistas y aplicar filtros por género y texto de búsqueda. Desde artistas.js también se permite consultar los Fan Packs asociados y agregarlos a las reservas mediante la función agregarFanPack() importada desde catalogo.js.
- **Módulo de eventos musicales (eventosMusicales.js):** Se importan los datos de los eventos y las funciones obtenerFavoritos(), alternarFavorito() e inicializarModuloEventos(). También se inicializa el módulo de eventos dentro del contenedor #contenedor-modulo-eventos.
- **Módulo de detalle de eventos (detalleEventos.js):** Se utiliza la función abrirDetalleEvento() para mostrar la información completa del evento seleccionado desde la lista de eventos guardados.
- **Sincronización de eventos favoritos:** Se incorporaron escuchas para los eventos favoritosActualizados y storage, permitiendo actualizar la lista de

eventos guardados cuando se modifican los favoritos. También se utiliza el evento storage para detectar cambios en los artistas seguidos y las reservas realizados desde otras pestañas.

Fundamentación Técnica de JavaScript (perfil.js)

Para la inicialización, organización y funcionamiento de la vista Perfil se utilizaron las siguientes características de JavaScript y métodos de la API del Modelo de Objetos del Documento (DOM):

- **Sentencia import:** Se utiliza para importar los datos fanPacks y artistasData, junto con funciones de los módulos catalogo.js, eventosMusicales.js, detalleEventos.js y detalleCatalogo.js.
- **Métodos document.querySelector(), document.querySelectorAll() y document.getElementById()**: Se utilizan para seleccionar elementos HTML específicos, como las grillas, la tabla de reservas, los filtros y las secciones que se muestran u ocultan.
- **Modelo de Objetos del Documento (DOM)**: Se utiliza para acceder y modificar elementos de la estructura HTML mediante propiedades como innerHTML, textContent, style.display y classList.
- **Definición de funciones:** Se definieron funciones como ocultarSecciones(), mostrarSeccion(), volverAlPerfil(), renderizarArtistas(), renderizarFanPacks(), actualizarReservas(), agregarPackDesdePerfil(), quitarPackDesdePerfil(), renderizarReservas() y renderizarEventosGuardados() para organizar las distintas operaciones de la vista.
- **Métodos de arreglos filter(), map(), find(), includes() y forEach():** Se utilizan para filtrar información, generar contenido dinámico, buscar artistas y Fan Packs, verificar coincidencias y recorrer los elementos de los arreglos.
- **Método addEventListener():** Se utiliza para escuchar cambios en los filtros, clics sobre botones y enlaces, envíos de formularios y actualizaciones relacionadas con los eventos guardados.
- **Funciones flecha:** Se utilizan en funciones callback, por ejemplo, al recorrer arreglos con forEach() y responder a eventos mediante addEventListener().

- **Propiedad dataset:** Se utiliza para recuperar los identificadores de los eventos y Fan Packs desde los atributos data-* de los botones.
- **Método closest()**: Permite identificar el botón o enlace correspondiente a partir del elemento sobre el que se realizó el clic.
- **Método preventDefault():** Se utiliza para evitar el comportamiento predeterminado de los enlaces y formularios cuando la navegación o el procesamiento deben ser controlados mediante JavaScript.
- **Método scrollIntoView():** Se utiliza para desplazar la página hasta la sección seleccionada, con un desplazamiento suave mediante behavior: "smooth" y alineación al comienzo mediante block: "start".
- **Objeto window:** Se utiliza para escuchar actualizaciones de favoritos y cambios en el almacenamiento mediante window.addEventListener(), y para actualizar el fragmento de la dirección mediante window.location.hash.
- **Métodos de almacenamiento localStorage.getItem(), localStorage.setItem() y localStorage.removeItem()**: Se utilizan para conservar los identificadores de artistas seguidos y los datos de favoritos y reservas. La gestión de las reservas se realiza centralizadamente desde catalogo.js.
- **Métodos split() y join():** Se utilizan para separar y unir los identificadores y datos almacenados en cadenas de texto mediante el carácter |.
- **Método toLocaleString():** Se utiliza para presentar los precios con formato numérico localizado para Argentina (es-AR).

Registro de Cambios y Modificaciones (artistas.html)

En relación con la maqueta estructural desarrollada en el TP3, se implementaron transformaciones profundas tanto en el código HTML como en la arquitectura de estilos y scripts para cumplir con los requerimientos de integración del TP4 y las correcciones del docente:

- Eliminación de Hardcodeo en el Catálogo: Se removieron por completo las tarjetas de artistas y las secciones de detalle estáticas del HTML. El contenedor #artistas-app ahora se pobla dinámicamente mediante JavaScript, cumpliendo con la consigna de renderizado vía script.

- Transición SPA gestionada por JavaScript: Se eliminó la lógica de CSS basada en las pseudoclases :target y :has(). Ahora, la navegación entre el catálogo y el detalle del artista se maneja manipulando la clase .seccion-oculta mediante JavaScript, logrando una experiencia de Single-Page Application (SPA) más robusta y controlada.
- Integración Inter-módulos: Se importó el array fanPacks desde el módulo catalogo.js (desarrollado por la compañera) para inyectar dinámicamente el Fan Pack correspondiente al artista seleccionado en la vista de detalle, evitando la duplicación de datos.

- Corrección de Organización CSS (Feedback TP3): Atendiendo a las observaciones del docente, se trasladaron todas las reglas genéricas y reutilizables (como .boton, .boton-volver, .detalle-hero, .panel-box) al archivo maestro estilos.css. El archivo artistas.css ahora contiene exclusivamente estilos específicos de esta vista, y se unificó toda la nomenclatura a BEM en castellano.

- Persistencia de Estado: Se implementó localStorage para que el botón "+ Seguir" recuerde la preferencia del usuario entre sesiones y se refleje automáticamente en la vista "Artistas Seguidos" del perfil.

Desarrollo y Codificación de la Vista Artistas (artistas.html, artistas.css y artistas.js)

La página central de exploración musical se transformó en una aplicación dinámica y modular:

- Renderizado Inicial: Al cargar la página, el script artistas.js invoca a renderizarArtistas() para poblar la grilla con los datos del módulo datosArtistas.js.

- Búsqueda y Filtrado en Tiempo Real: Se implementaron Event Listeners en el campo de búsqueda y los filtros de género. Utilizando los métodos .filter() e .includes() de JavaScript, la grilla se actualiza instantáneamente sin recargar la página. Si no hay coincidencias, se muestra un mensaje amigable de "estado vacío" (Empty State).

- Navegación SPA Dinámica: Al hacer clic en "Ver detalles", la función mostrarDetalleArtista(id) busca los datos del artista, inyecta la información (incluyendo el setlist y los horarios) en los elementos del DOM con IDs específicos, consulta el módulo catalogo.js para renderizar el Fan Pack asociado, y finalmente oculta el catálogo y muestra el detalle manipulando la clase .seccion-oculta.

- Interacción de Usuario: Se agregó funcionalidad al botón "+ Seguir", que alterna su estado visual y guarda el nombre del artista en el localStorage del navegador.

Fundamentación Técnica de Propiedades y Métodos JavaScript (MDN / Documentación Oficial)

Para la maquetación dinámica de esta vista se aplicaron herramientas de JavaScript moderno (ES6+) alineadas con la documentación oficial de MDN Web Docs:

- Módulos ES6 (import / export):
  Uso en el proyecto: Permiten dividir el código en archivos lógicos e independientes (datosArtistas.js, funcionesArtistas.js, catalogo.js), mejorando la legibilidad, el mantenimiento y evitando la contaminación del ámbito global (scope).

- Array.prototype.filter() y Array.prototype.includes():
  Uso en el proyecto: Utilizados en la función filtrarArtistas() para crear un nuevo arreglo con los elementos que coinciden con el género seleccionado y el texto de búsqueda, sin mutar el arreglo original de datos.

- Element.classList.add() / Element.classList.remove():
  Uso en el proyecto: Mecanismo principal para gestionar la visibilidad de las secciones (comportamiento SPA), alternando la clase .seccion-oculta (definida en estilos.css con display: none) en lugar de depender de pseudoclases de CSS.

- Window.localStorage (getItem / setItem):
  Uso en el proyecto: Empleados para persistir el estado del botón "Seguir". Los datos se convierten a formato JSON (JSON.stringify / JSON.parse) para guardar un arreglo de nombres de artistas, sincronizando esta información con la vista de "Artistas Seguidos" en el perfil.

- Delegación de Eventos (Event.target.closest()):
  Uso en el proyecto: Se utiliza un único Event Listener en el document para detectar clics en elementos generados dinámicamente (como los botones "Ver detalles" de las tarjetas), mejorando el rendimiento y evitando la pérdida de referencias al re-renderizar el DOM.

Registro de Cambios y Modificaciones (contacto.html)

En relación con la maqueta desarrollada en el TP3, se adaptó la interfaz para corregir el comportamiento del formulario y unificar la organización del código:

- Intercepción del Envío del Formulario vía JavaScript: Se eliminó el comportamiento por defecto del formulario (que generaba una página de error "Cannot POST" al no existir un backend). Ahora, JavaScript intercepta el evento submit, previene la recarga de la página y muestra un mensaje de confirmación dinámico.

- Unificación de Estilos Genéricos (Feedback TP3): Se trasladaron selectores genéricos (ej. .boton, .campo-grupo) desde contacto.css hacia estilos.css, cumpliendo con el principio de separación de responsabilidades. contacto.css ahora solo contiene reglas específicas de esta vista.

- Mejora en la Accesibilidad y UX: Se potenció el uso del contenedor #id-contacto-dialogo con role="alert", trabajando en sinergia con la pseudoclase CSS :empty para mostrar y ocultar mensajes de estado de forma fluida y accesible para lectores de pantalla.

Desarrollo y Codificación de la Vista Contacto (contacto.html, contacto.css y contacto.js)

- La pantalla se organizó en módulos semánticos claros, con un enfoque en la interacción sin recarga de página:
- Formulario de Soporte y Propuestas: Maquetado con etiquetas explícitas (<label for="...">) y validaciones nativas de HTML5 (required, minlength). Al presionar "Enviar", el script contacto.js ejecuta evento.preventDefault(), inyecta un mensaje de éxito en el contenedor de alertas, limpia los campos del formulario mediante .reset() y programa la ocultación automática del mensaje a los 5 segundos.
- Paneles de Redes Sociales y Localización: Se estructuraron dos tarjetas (.panel-box) que agrupan los enlaces oficiales y la dirección física institucional junto a un mapa interactivo embebido (<iframe>), manteniendo la grilla responsiva de 2 columnas en escritorio.

Fundamentación Técnica de Propiedades y Métodos (MDN / TID)

- Event.preventDefault():
  Uso en el proyecto: Aplicado en el evento submit del formulario de contacto para anular el comportamiento por defecto del navegador (recarga o salto de ancla), permitiendo que JavaScript controle el flujo de la aplicación y muestre el mensaje de éxito sin redirigir a una página de error.

- HTMLFormElement.reset():
  Uso en el proyecto: Utilizado inmediatamente después de mostrar el mensaje de éxito para limpiar todos los campos del formulario, mejorando la experiencia de usuario (UX) y preparando el formulario para un nuevo envío.

- Pseudoclase :empty (Sinergia con JS):
  Uso en el proyecto: Declara display: none; sobre .contacto-mensaje:empty en el CSS. Cuando JavaScript inyecta texto en el div, este deja de estar vacío y se muestra automáticamente. Cuando setTimeout limpia el texto, vuelve a estar :empty y se oculta, evitando lógica de visualización compleja en JS.

- Window.setTimeout():
  Uso en el proyecto: Empleado para ejecutar una función que limpia el contenido del mensaje de alerta después de 5000 milisegundos (5 segundos), brindando feedback temporal al usuario sin intervención manual.
  Atributo ARIA role="alert":
  Uso en el proyecto: Asignado al contenedor #id-contacto-dialogo. Establece un canal prioritario que notifica a las tecnologías asistivas (lectores de pantalla) en el instante en que JavaScript inyecta el mensaje de éxito, sin requerir que el usuario mueva el foco.

Fuentes Consultadas

- Colegio Universitario IES Siglo 21. (2024). Material Didáctico Institucional — Unidad 1 (Escenarios Profesionales 1 y 2). Materia de Aplicaciones Web I.
- MDN Web Docs. (s. f.). ::before. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/::before
- MDN Web Docs. (s. f.). :hover. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/:hover
- MDN Web Docs. (s. f.). ARIA alert role. Mozilla Developer Network.
- MDN Web Docs. (s. f.). Array.prototype.filter(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- MDN Web Docs. (s. f.). Array.prototype.includes(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/includes
- MDN Web Docs. (s. f.). Attribute selectors. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/Attribute_selectors
- MDN Web Docs. (s. f.). border-radius. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/border-radius
- MDN Web Docs. (s. f.). box-sizing. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/box-sizing
- MDN Web Docs. (s. f.). Child combinator (>). Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Selectors/Child_combinator
- MDN Web Docs. (s. f.). CSS Grid Layout - repeat(). Mozilla Developer Network.
- MDN Web Docs. (s. f.). CSS position property. Mozilla Developer Network.

- MDN Web Docs. (s. f.). display. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/display
- MDN Web Docs. (s. f.). Element.classList. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Element/classList
- MDN Web Docs. (s. f.). Event.preventDefault(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Event/preventDefault
- MDN Web Docs. (s. f.). flex-wrap. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/flex-wrap
- MDN Web Docs. (s. f.). HTML Standard & Accessibility Guidelines. Mozilla Developer Network.
- MDN Web Docs. (s. f.). HTMLFormElement.reset(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/HTMLFormElement/reset
- MDN Web Docs. (s. f.). linear-gradient(). Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/gradient/linear-gradient
- MDN Web Docs. (s. f.). overflow. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/overflow
- MDN Web Docs. (s. f.). Pseudo-elements. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Pseudo-elements
- MDN Web Docs. (s. f.). The :empty pseudo-class. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/CSS/:empty
- MDN Web Docs. (s. f.). import. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/import
- MDN Web Docs. (s. f.). Document.querySelector(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Document/querySelector

- MDN Web Docs. (s. f.). Document Object Model (DOM). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
- MDN Web Docs. (s. f.). Object. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object
- MDN Web Docs. (s. f.). export. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Statements/export
- MDN Web Docs. (s. f.). JavaScript functions. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Functions
- MDN Web Docs. (s. f.). isNaN(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/isNaN
- MDN Web Docs. (s. f.). Array.prototype.map(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/map
- MDN Web Docs. (s. f.). Array.prototype.push(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/push
- MDN Web Docs. (s. f.). Array.prototype.splice(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/splice
- MDN Web Docs. (s. f.). Event. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/API/Event/Event
- MDN Web Docs. (s. f.). EventTarget.dispatchEvent(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/EventTarget/dispatchEvent

- MDN Web Docs. (s. f.). Element.innerHTML. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Element/innerHTML
- MDN Web Docs. (s. f.). Element.querySelectorAll(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/API/Element/querySelectorAll
- MDN Web Docs. (s. f.). Element.scrollIntoView(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Element/scrollIntoView
- MDN Web Docs. (s. f.). Arrow functions. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Functions/Arrow_functions
- MDN Web Docs. (s. f.). Array.prototype.forEach(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array/forEach
- MDN Web Docs. (s. f.). Spread syntax. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/Spread_syntax
- MDN Web Docs. (s. f.). EventTarget.addEventListener(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
- MDN Web Docs. (s. f.). The :focus pseudo-class. Mozilla Developer Network.
- MDN Web Docs. (s. f.). The :has() relational pseudo-class. Mozilla Developer Network.
- MDN Web Docs. (s. f.). The :target pseudo-class. Mozilla Developer Network.
- MDN Web Docs. (s. f.). The HTML <form> element & POST method. Mozilla Developer Network.
- MDN Web Docs. (s. f.). The object-fit property. Mozilla Developer Network.

- MDN Web Docs. (s. f.). transition. Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/transition
- MDN Web Docs. (s. f.). Using CSS custom properties (variables). Mozilla Developer Network. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Cascading_variables
- MDN Web Docs. (s. f.). Window.localStorage. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Window/localStorage
- MDN Web Docs. (s. f.). Window.setTimeout(). Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/API/Window/setTimeout
- MDN Web Docs. (s. f.). Working with JavaScript modules. Mozilla Developer Network. Recuperado de: https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Modules
- W3C. (2014). HTML5 Semantics and Structure Guidelines. World Wide Web Consortium.
- W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1 Recommendations. World Wide Web Consortium.